# DisarmCache 实验项目交接手册

这份文档用于在新的对话窗口继续实验，包含当前代码状态、服务器路径、4SAPI 配置、NQ 数据集和 B2/B3 运行步骤。

## 当前状态

GitHub 仓库：https://github.com/5ijj-2022/lunwen

服务器项目目录：

```
/data1/lab409/wwj/lunwen-main-git
```

实验目录：

```
/data1/lab409/wwj/lunwen-main-git/experiments/disarmcache
```

模型目录：

```
/data1/lab409/wwj/models
```

已完成：

- Contriever 干净检索器加载。
- NQ 知识库加载，当前约 10,113 篇文档，测试问题 100 条。
- 语义缓存持久化。
- B2：clean retriever + cache-first。
- B3：clean retriever + retrieval-first。
- 4SAPI OpenAI 兼容接口。
- 查询、检索、缓存、批量和绘图脚本。
- API 错误详细记录。
- API_KEY_ENV、API_BASE_URL 环境变量兼容。
- query_pretty.py 清晰显示单条查询。

最近推送到 main 的提交：

```
d949218 feat: improve readable query responses
aadb110 fix: honor API_KEY_ENV and API_BASE_URL
```

## 服务器启动环境

每次重新连接服务器后：

```
source /data1/anaconda3/etc/profile.d/conda.sh
conda activate disarmcache
cd /data1/lab409/wwj/lunwen-main-git/experiments/disarmcache
```

检查 GPU：

```
python3 - <<'PY'
import torch
print("torch:", torch.__version__)
print("CUDA:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
PY
```

模型检查：

```
test -d /data1/lab409/wwj/models/contriever && echo "Contriever 正常"
test -d /data1/lab409/wwj/models/all-MiniLM-L6-v2 && echo "缓存编码器正常"
```

# 4SAPI 配置

正确地址是：

```
https://4sapi.org/v1
```

不能写成 Markdown 链接：

```
[https://4sapi.org/v1](https://4sapi.org/v1)
```

启动服务的同一个终端中执行：

```
export API_BASE_URL="https://4sapi.org/v1"
export API_KEY_ENV="API_SECRET_KEY"
export GENERATOR_BACKEND="openai"
export GENERATOR_MODEL="gpt-4o-mini"
read -rsp "请输入纯 API 密钥: " API_SECRET_KEY
echo
export API_SECRET_KEY
```

检查但不要打印密钥：

```
python3 - <<'PY'
import os
print("BASE_URL:", repr(os.environ.get("API_BASE_URL")))
print("API key 已配置:", bool(os.environ.get("API_SECRET_KEY")))
PY
```

如果密钥曾经出现在聊天或截图中，应先在 4SAPI 控制台撤销并重新生成。

# NQ 数据集结构

```
data/datasets/nq/
├── corpus/clean.jsonl
└── questions/test.jsonl
```

以后新增数据集，只需复制这个目录结构：

```
data/datasets/<dataset-name>/corpus/clean.jsonl
data/datasets/<dataset-name>/questions/test.jsonl
```

不需要复制代码，运行时把 nq 换成新的数据集名称。

# 启动 B2

先停止旧的 8080 服务：

```
PID=$(ss -ltnp | awk -F'pid=' '/127.0.0.1:8080/ {split($2,a,","); print a[1]; exit}')
if [ -n "$PID" ]; then
  ps -fp "$PID"
  kill "$PID"
  sleep 2
fi
```

配置 API 后启动：

```
cd /data1/lab409/wwj/lunwen-main-git/experiments/disarmcache
./start_experiment.sh nq B2 8080
```

B2 是 cache-first：先查语义缓存，命中则直接返回，未命中才检索和生成。

检查：

```
curl -sS http://127.0.0.1:8080/health | python3 -m json.tool
```

应看到：

```
"query_order": "cache-first"
```

# 清晰显示单条查询

推荐：

```
python3 query_pretty.py \
  "What is the capital of France?" \
  --url http://127.0.0.1:8080/query
```

第一次通常显示 MISS、检索执行 True、生成执行 True。相同问题第二次通常显示 HIT、检索执行 False、生成执行 False。

也可以使用原始 curl，但不要加 -i：

```
curl -sS --max-time 300 \
  -X POST http://127.0.0.1:8080/query \
  -H 'Content-Type: application/json' \
  -d '{"query":"What is the capital of France?"}' \
  | python3 -m json.tool
```

-i 会把 HTTP 头和 JSON 混在一起，导致终端显示很乱。

# B2 批量实验

```
export DATA_ROOT="$PWD/data/datasets"
export RUN_ROOT="$PWD/runs/real"
```

```
./run_experiment.sh \
  nq B2 \
  "$DATA_ROOT/nq/questions/test.jsonl" \
```

```
http://127.0.0.1:8080/query
```

输出位置：

```
runs/real/nq/B2/cache.jsonl
runs/real/nq/B2/events.jsonl
runs/real/nq/B2/test.responses.jsonl
runs/real/nq/B2/test.responses.summary.json
```

绘图：

```
python3 plot_results.py \
  runs/real/nq/B2/events.jsonl \
  --out runs/real/nq/B2/figures
```

# B3 实验

停止 B2 服务，重新配置 API 环境变量，然后执行：

```
./start_experiment.sh nq B3 8080
```

B3 是 retrieval-first：先检索，再检查缓存，最后返回缓存答案或调用生成模型。

检查：

```
curl -sS http://127.0.0.1:8080/health | python3 -m json.tool
```

应看到：

```
"query_order": "retrieval-first"
```

批量：

```
./run_experiment.sh \
  nq B3 \
  "$DATA_ROOT/nq/questions/test.jsonl" \
  http://127.0.0.1:8080/query
```

绘图：

```
python3 plot_results.py \
  runs/real/nq/B3/events.jsonl \
  --out runs/real/nq/B3/figures
```

# 查看检索和缓存

只看检索：

```
curl -sS -X POST http://127.0.0.1:8080/retrieve \
  -H 'Content-Type: application/json' \
  -d '{"query":"What is the capital of France?"}' \
  | python3 -m json.tool
```

查看缓存：

```
curl -sS http://127.0.0.1:8080/cache | python3 -m json.tool
```

B2 命中缓存时 /query 可能不执行检索，这是设计行为；查看当前检索器输出请使用 /retrieve。

## 故障排查

检查正在运行的服务环境：

```
PID=$(ss -ltnp | awk -F'pid=' '/127.0.0.1:8080/ {split($2,a,","); print a[1]; exit}')
tr '\0' '\n' < /proc/"$PID"/environ | grep -E '^(API_BASE_URL|API_KEY_ENV|GENERATOR_MODEL)='
```

正确值应为：

```
API_BASE_URL=https://4sapi.org/v1
API_KEY_ENV=API_SECRET_KEY
GENERATOR_MODEL=gpt-4o-mini
```

如果返回旧的 Connection error，说明 8080 仍是旧进程，停止后重新启动。批量运行前先用 query_pretty.py 成功测试一条。端口占用时可以使用 8081，并把查询地址改为 http://127.0.0.1:8081/query。

当前仓库没有 test_generator_client.py，不要运行它；使用 query_pretty.py 或 /query 检查 API。

## 后续计划

```
runs/real/nq/B2/
runs/real/nq/B3/
runs/real/<other-dataset>/B2/
runs/real/<other-dataset>/B3/
```

后续完成 B4（联合缓存）、B5（编辑后检索器无缓存）、B6（编辑后检索器 cache-first）、B7（编辑后检索器 retrieval-first），并比较命中率、检索分数、生成次数、准确率、目标错误率和延迟。

## 新对话窗口上下文

我正在继续 /data1/lab409/wwj/lunwen-main-git 的 DisarmCache 实验。代码已更新到 GitHub main 的提交 d949218。服务器使用 Conda 环境 disarmcache，模型目录为 /data1/lab409/wwj/models，4SAPI 地址是 https://4sapi.org/v1，模型是 gpt-4o-mini，密钥变量是 API_SECRET_KEY。NQ 数据位于 experiments/disarmcache/data/datasets/nq，知识库约 10,113 篇文档，测试集 100 条。B2 使用 ./start_experiment.sh nq B2 8080，B3 使用 ./start_experiment.sh nq B3 8080。单条查询使用 python3 query_pretty.py "问题" --url http://127.0.0.1:8080/query。请继续帮我完成 NQ 的 B2/B3 对比、结果统计和后续 B4-B7 实验，不要索要或输出 API 密钥。